```python
import streamlit as st
import yfinance as yf
import pandas as pd
import numpy as np
import plotly.graph_objects as go


##
## NEON COLOR PALETTE
##

NEON_PINK = "#FF2D95"
NEON_BLUE = "#00D9FF"
NEON_GREEN = "#39FF14"
NEON_PURPLE = "#BF5AF2"
NEON_ORANGE = "#FF9F0A"


##
## VALUATION DASHBOARD
##

st.set_page_config(
    page_title="Valuation Dashboard",
    layout="wide"
)

st.title("Valuation Dashboard")
st.write("DCF, Comparable Company Analysis, and Sensitivity Analysis")


##
## TABS
##

fundamentals_tab, dcf_tab, comps_tab, sensitivity_tab, extension_tab, analysis_tab
    "Fundamentals & FCF",
    "DCF Valuation",
    "Comparable Companies",
    "Sensitivity Analysis",
    "Valuation Summary",
    "Investment Analysis"
])


##
## HELPER FUNCTIONS
##

# Checks multiple possible yfinance labels so missing or renamed
# statement items are less likely to break the dashboard.
def pick(df, *labels, default=np.nan):
```

```
    for label in labels:
        if label in df.columns:
            return df[label]
    return default


# Safely retrieves values from yfinance company information.
def safe_info(info, key, default=np.nan):
    value = info.get(key, default)

    if value is None:
        return default

    return value


##
## COMPANY SETTINGS
##

st.sidebar.header("Company Settings")

ticker = st.sidebar.text_input(
    "Ticker",
    value="LULU"
).upper()

if not ticker.strip():
    st.error("Enter a ticker symbol.")
    st.stop()


##
## DOWNLOAD FINANCIAL DATA
##

# Download financial statements from yfinance.
# Successful downloads are also saved as local CSV files.
# These CSV files provide a backup if yfinance is unavailable later.
@st.cache_data(ttl=300)
def load_financial_data(symbol):

    ##
    ## DOWNLOAD FINANCIAL STATEMENTS
    ##

    income = pd.DataFrame()
    balance = pd.DataFrame()
    cashflow = pd.DataFrame()

    # Try yfinance more than once because it can occasionally
    # return an empty statement even when the ticker is valid.
```

```
for attempt in range(3):

    try:

        stock = yf.Ticker(symbol)

        income = stock.income_stmt.T.sort_index()
        balance = stock.balance_sheet.T.sort_index()
        cashflow = stock.cashflow.T.sort_index()

        # Only accept the download when all three statements exist.
        if (
            not income.empty
            and not balance.empty
            and not cashflow.empty
        ):

            # Automatically save successful downloads as backups.
            income.to_csv(f"{symbol}_income.csv")
            balance.to_csv(f"{symbol}_balance.csv")
            cashflow.to_csv(f"{symbol}_cashflow.csv")

            data_source = "yfinance"
            break

    except Exception:
        pass

##
## CSV FALLBACK
##

# If yfinance still failed after the retries,
# try a previously created backup for that ticker.
if (
    income.empty
    or balance.empty
    or cashflow.empty
):

    try:

        income = pd.read_csv(
            f"{symbol}_income.csv",
            index_col=0,
            parse_dates=True
        )

        balance = pd.read_csv(
            f"{symbol}_balance.csv",
            index_col=0,
            parse_dates=True
```

```
            )

            cashflow = pd.read_csv(
                f"{symbol}_cashflow.csv",
                index_col=0,
                parse_dates=True
            )

            data_source = "CSV Backup"

        except Exception:

            return (
                pd.DataFrame(),
                pd.DataFrame(),
                pd.DataFrame(),
                {},
                "Unavailable"
            )


    ##
    ## MARKET DATA
    ##

    # Request market data separately so an info error does not
    # discard financial statements that loaded successfully.
    try:

        stock = yf.Ticker(symbol)
        info = stock.info

    except Exception:

        info = {}

    return (
        income,
        balance,
        cashflow,
        info,
        data_source
    )

income, balance, cashflow, info, data_source = load_financial_data(ticker)


##
## CHECK FINANCIAL DATA
##

if income.empty or balance.empty or cashflow.empty:
```

```
    st.error(
        "Financial statements could not be downloaded and "
        "no cached CSV backup is available for this ticker."
    )

    st.stop()


# Show where the financial statement data came from.
if data_source == "CSV Backup":

    st.warning(
        "yfinance financial statements were unavailable. "
        "Cached CSV financial statements are being used."
    )



##
## PART 1 - FUNDAMENTALS AND FREE CASH FLOW
##

# Pull important income statement items.
revenue = pick(
    income,
    "Total Revenue",
    "Operating Revenue"
)

net_income = pick(
    income,
    "Net Income",
    "Net Income Common Stockholders"
)

operating_income = pick(
    income,
    "Operating Income"
)


# Pull cash flow statement items.
operating_cf = pick(
    cashflow,
    "Operating Cash Flow",
    "Total Cash From Operating Activities"
)

capex = pick(
    cashflow,
    "Capital Expenditure",
```

```
    "Capital Expenditures"
)


# Free Cash Flow = Operating Cash Flow - Capital Expenditures.
# abs() is used because CapEx is commonly reported as a negative outflow.
free_cash_flow = operating_cf - abs(capex)


# Combine major fundamentals into one table.
fundamentals = pd.DataFrame({
    "Revenue": revenue,
    "Net Income": net_income,
    "Operating Income": operating_income,
    "Operating Cash Flow": operating_cf,
    "Capital Expenditures": capex,
    "Free Cash Flow": free_cash_flow
})

fundamentals = fundamentals.sort_index().tail(4)

fundamentals_billions = fundamentals / 1_000_000_000

# Display fiscal year instead of full date.
fundamentals_billions.index = fundamentals_billions.index.year
fundamentals_billions.index.name = "Fiscal Year"


##
## FINANCIAL RATIOS
##

# Profitability ratios measure profit generated from revenue.
net_margin = net_income / revenue
operating_margin = operating_income / revenue


current_assets = pick(
    balance,
    "Current Assets",
    "Total Current Assets"
)

current_liabilities = pick(
    balance,
    "Current Liabilities",
    "Total Current Liabilities"
)

total_debt_series = pick(
    balance,
    "Total Debt"
```

```
)

stockholders_equity = pick(
    balance,
    "Stockholders Equity",
    "Total Stockholder Equity"
)


# Liquidity and leverage ratios.
current_ratio = current_assets / current_liabilities
debt_to_equity = total_debt_series / stockholders_equity


ratios = pd.DataFrame({
    "Net Margin": net_margin,
    "Operating Margin": operating_margin,
    "Current Ratio": current_ratio,
    "Debt-to-Equity": debt_to_equity
})

ratios = ratios.sort_index().tail(4)

ratio_display = ratios.copy()

ratio_display["Net Margin"] = ratio_display["Net Margin"] * 100
ratio_display["Operating Margin"] = ratio_display["Operating Margin"] * 100

ratio_display.index = ratio_display.index.year
ratio_display.index.name = "Fiscal Year"


##
## DISPLAY PART 1
##

with fundamentals_tab:

    st.subheader(f"{ticker} - Fundamentals & Free Cash Flow")

    st.write("Financial Statement Fundamentals ($ Billions)")

    st.dataframe(
        fundamentals_billions.style.format("{:.2f}")
    )


    st.write("Financial Ratios")

    st.dataframe(
        ratio_display.style.format({
            "Net Margin": "{:.2f}%",
```

```
            "Operating Margin": "{:.2f}%",
            "Current Ratio": "{:.2f}",
            "Debt-to-Equity": "{:.2f}"
        })
    )


    st.write("Historical Free Cash Flow ($ Billions)")

    fcf_chart = go.Figure()

    fcf_chart.add_trace(
        go.Bar(
            x=fundamentals_billions.index.astype(str),
            y=fundamentals_billions["Free Cash Flow"],
            name="Free Cash Flow",
            marker_color=NEON_PINK
        )
    )

    fcf_chart.update_layout(
        xaxis_title="Fiscal Year",
        yaxis_title="Free Cash Flow ($ Billions)",
        template="plotly_dark",
        xaxis=dict(type="category")
    )

    st.plotly_chart(
        fcf_chart,
        use_container_width=True
    )


##
## PART 2 — DCF VALUATION
##

# Professor requires a 3-year average FCF as the DCF starting point.
base_fcf = (
    free_cash_flow
    .dropna()
    .sort_index()
    .tail(3)
    .mean()
)


##

## MARKET DATA
##
```

```
beta = safe_info(info, "beta")
market_cap = safe_info(info, "marketCap")
current_price = safe_info(info, "currentPrice")
shares_outstanding = safe_info(info, "sharesOutstanding")


# Most recent debt.
debt_clean = total_debt_series.dropna()

if debt_clean.empty:
    total_debt = 0
else:
    total_debt = debt_clean.iloc[-1]


# Most recent cash.
cash_series = pick(
    balance,
    "Cash Cash Equivalents And Short Term Investments",
    "Cash And Cash Equivalents",
    "Cash"
)

cash_clean = cash_series.dropna()

if cash_clean.empty:
    cash = 0
else:
    cash = cash_clean.iloc[-1]


# Stop the DCF if essential market data is unavailable.
if (
    pd.isna(beta)
    or pd.isna(market_cap)
    or pd.isna(current_price)
    or pd.isna(shares_outstanding)
):

    st.error(
        "Required market data is unavailable for this ticker. "
        "The DCF cannot be completed."
    )

    st.stop()


##
## WACC ASSUMPTIONS
##

st.sidebar.header("DCF Assumptions")
```

```
risk_free_rate = st.sidebar.number_input(
    "Risk-Free Rate (%)",
    value=4.0,
    step=0.1
) / 100

market_return = st.sidebar.number_input(
    "Expected Market Return (%)",
    value=10.0,
    step=0.1
) / 100

cost_of_debt = st.sidebar.number_input(
    "Pre-Tax Cost of Debt (%)",
    value=5.0,
    step=0.1
) / 100

tax_rate = st.sidebar.number_input(
    "Tax Rate (%)",
    value=21.0,
    step=0.1
) / 100


##
## CAPM AND WACC
##

# CAPM: Cost of Equity = Rf + Beta(Rm - Rf)
cost_of_equity = (
    risk_free_rate
    + beta * (market_return - risk_free_rate)
)


# Market-value capital structure weights.
total_capital = market_cap + total_debt

equity_weight = market_cap / total_capital
debt_weight = total_debt / total_capital


# WACC includes the tax benefit of debt financing.
wacc = (
    equity_weight * cost_of_equity
    + debt_weight * cost_of_debt * (1 - tax_rate)
)


##
```

```
## DCF FORECAST ASSUMPTIONS
##

forecast_growth = st.sidebar.number_input(
    "Forecast FCF Growth (%)",
    value=8.0,
    step=0.5
) / 100

terminal_growth = st.sidebar.number_input(
    "Terminal Growth (%)",
    value=2.5,
    step=0.1
) / 100

forecast_years = st.sidebar.number_input(
    "Forecast Years",
    min_value=1,
    max_value=10,
    value=5,
    step=1
)


# User adjusts WACC around the calculated CAPM baseline.
wacc_adjustment = st.sidebar.slider(
    "WACC Adjustment (%)",
    min_value=-3.0,
    max_value=3.0,
    value=0.0,
    step=0.1
) / 100

adjusted_wacc = wacc + wacc_adjustment


##
## DCF VALIDATION
##

# Gordon Growth requires terminal growth below WACC.
if terminal_growth >= adjusted_wacc:

    st.error(
        "Terminal growth must be less than WACC."
    )

    st.stop()


##
## PROJECT FREE CASH FLOW
```

```
##

years = np.arange(
    1,
    forecast_years + 1
)

projected_fcf = (
    base_fcf
    * (1 + forecast_growth) ** years
)

pv_fcf = (
    projected_fcf
    / (1 + adjusted_wacc) ** years
)


##
## TERMINAL VALUE
##

terminal_value = (
    projected_fcf[-1]
    * (1 + terminal_growth)
    / (adjusted_wacc - terminal_growth)
)

pv_terminal = (
    terminal_value
    / (1 + adjusted_wacc) ** forecast_years
)


##
## ENTERPRISE AND EQUITY VALUE
##

enterprise_value = (
    pv_fcf.sum()
    + pv_terminal
)

equity_value = (
    enterprise_value
    + cash
    - total_debt
)

intrinsic_value = (
    equity_value
    / shares_outstanding
```

```
)

upside_downside = (
    (intrinsic_value / current_price) - 1
) * 100

terminal_value_percent = (
    pv_terminal / enterprise_value
) * 100


##
## DCF FORECAST TABLE
##

dcf_forecast = pd.DataFrame({
    "Year": years,
    "Projected FCF ($B)": projected_fcf / 1_000_000_000,
    "Present Value ($B)": pv_fcf / 1_000_000_000
})


##
## DISPLAY PART 2
##

with dcf_tab:

    st.subheader(f"{ticker} - DCF Valuation")

    st.write("DCF Starting Point")

    col1, col2, col3, col4 = st.columns(4)

    col1.metric(
        "3-Year Average FCF",
        f"${base_fcf / 1_000_000_000:.2f}B"
    )

    col2.metric(
        "Beta",
        f"{beta:.2f}"
    )

    col3.metric(
        "Cost of Equity",
        f"{cost_of_equity:.2%}"
    )

    col4.metric(
        "Baseline WACC",
        f"{wacc:.2%}"
```

```
)


st.write("WACC Calculation")

wacc_table = pd.DataFrame({
    "Input": [
        "Risk-Free Rate",
        "Expected Market Return",
        "Beta",
        "Cost of Equity",
        "Pre-Tax Cost of Debt",
        "Tax Rate",
        "Equity Weight",
        "Debt Weight",
        "WACC"
    ],
    "Value": [
        f"{risk_free_rate:.2%}",
        f"{market_return:.2%}",
        f"{beta:.2f}",
        f"{cost_of_equity:.2%}",
        f"{cost_of_debt:.2%}",
        f"{tax_rate:.2%}",
        f"{equity_weight:.2%}",
        f"{debt_weight:.2%}",
        f"{wacc:.2%}"
    ]
})

st.dataframe(
    wacc_table,
    hide_index=True
)


st.write("DCF Assumptions")

assumption_table = pd.DataFrame({
    "Assumption": [
        "Forecast FCF Growth",
        "Terminal Growth",
        "Baseline WACC",
        "WACC Adjustment",
        "Adjusted WACC",
        "Forecast Years"
    ],
    "Value": [
        f"{forecast_growth:.2%}",
        f"{terminal_growth:.2%}",
        f"{wacc:.2%}",
        f"{wacc_adjustment:.2%}",
```

```
        f"{adjusted_wacc:.2%}",
        str(forecast_years)
    ]
})

st.dataframe(
    assumption_table,
    hide_index=True
)


st.write("Projected Free Cash Flow")

st.dataframe(
    dcf_forecast.style.format({
        "Year": "{:.0f}",
        "Projected FCF ($B)": "${:.2f}",
        "Present Value ($B)": "${:.2f}"
    }),
    hide_index=True
)


st.write("DCF Valuation Results")

col1, col2, col3 = st.columns(3)

col1.metric(
    "Intrinsic Value / Share",
    f"${intrinsic_value:.2f}"
)

col2.metric(
    "Current Market Price",
    f"${current_price:.2f}"
)

col3.metric(
    "Implied Upside / Downside",
    f"{upside_downside:.2f}%"
)


col4, col5, col6 = st.columns(3)

col4.metric(
    "Enterprise Value",
    f"${enterprise_value / 1_000_000_000:.2f}B"
)

col5.metric(
    "Equity Value",
```

```
        f"${equity_value / 1_000_000_000:.2f}B"
    )

    col6.metric(
        "Terminal Value % of EV",
        f"{terminal_value_percent:.2f}%"
    )


##
## PART 3 - COMPARABLE COMPANY ANALYSIS
##

st.sidebar.header("Comparable Companies")

# Default peers are selected for LULU, but the user can replace them
# when analyzing another company.
peer_input = st.sidebar.text_input(
    "Peer Tickers",
    value="NKE, DECK, RL"
)

peer_tickers = [
    peer.strip().upper()
    for peer in peer_input.split(",")
    if peer.strip()
]


##
## DOWNLOAD PEER DATA
##

peer_data = []

for peer in peer_tickers:

    try:

        peer_stock = yf.Ticker(peer)
        peer_info = peer_stock.info

        peer_data.append({
            "Ticker": peer,
            "P/E": peer_info.get(
                "trailingPE",
                np.nan
            ),
            "EV/EBITDA": peer_info.get(
                "enterpriseToEbitda",
                np.nan
            ),
```

```
            "P/S": peer_info.get(
                "priceToSalesTrailing12Months",
                np.nan
            )
        })

    except Exception:

        peer_data.append({
            "Ticker": peer,
            "P/E": np.nan,
            "EV/EBITDA": np.nan,
            "P/S": np.nan
        })


peer_table = pd.DataFrame(peer_data)


##
## PEER MEDIANS
##

median_pe = peer_table["P/E"].median()
median_ev_ebitda = peer_table["EV/EBITDA"].median()
median_ps = peer_table["P/S"].median()


##
## COMPANY METRICS FOR COMPS
##

eps = safe_info(
    info,
    "trailingEps"
)

ebitda = safe_info(
    info,
    "ebitda"
)

total_revenue = safe_info(
    info,
    "totalRevenue"
)


##
## IMPLIED VALUES
##
```

```
# P/E implied share value.
if pd.isna(median_pe) or pd.isna(eps):
    implied_pe = np.nan
else:
    implied_pe = median_pe * eps


# EV/EBITDA implied share value.
if pd.isna(median_ev_ebitda) or pd.isna(ebitda):

    implied_ev_ebitda = np.nan

else:

    implied_ev = (
        median_ev_ebitda
        * ebitda
    )

    implied_equity_ev = (
        implied_ev
        + cash
        - total_debt
    )

    implied_ev_ebitda = (
        implied_equity_ev
        / shares_outstanding
    )


# P/S implied share value.
if pd.isna(median_ps) or pd.isna(total_revenue):

    implied_ps = np.nan

else:

    implied_equity_ps = (
        median_ps
        * total_revenue
    )

    implied_ps = (
        implied_equity_ps
        / shares_outstanding
    )


implied_values = pd.Series([
    implied_pe,
    implied_ev_ebitda,
```

```
    implied_ps
]).dropna()

if implied_values.empty:
    comps_implied_value = np.nan
else:
    comps_implied_value = implied_values.mean()


##
## DISPLAY PART 3
##

with comps_tab:

    st.subheader(
        f"{ticker} - Comparable Company Analysis"
    )

    st.write("Peer Multiples")

    st.dataframe(
        peer_table.style.format({
            "P/E": "{:.2f}",
            "EV/EBITDA": "{:.2f}",
            "P/S": "{:.2f}"
        }),
        hide_index=True
    )


    st.write("Median Peer Multiples")

    median_table = pd.DataFrame({
        "Multiple": [
            "P/E",
            "EV/EBITDA",
            "P/S"
        ],
        "Median": [
            median_pe,
            median_ev_ebitda,
            median_ps
        ]
    })

    st.dataframe(
        median_table.style.format({
            "Median": "{:.2f}"
        }),
        hide_index=True
    )
```

```
st.write("Implied Share Values")

implied_table = pd.DataFrame({
    "Valuation Method": [
        "P/E",
        "EV/EBITDA",
        "P/S"
    ],
    "Implied Value / Share": [
        implied_pe,
        implied_ev_ebitda,
        implied_ps
    ]
})

st.dataframe(
    implied_table.style.format({
        "Implied Value / Share": "${:.2f}"
    }),
    hide_index=True
)


st.write("Valuation Comparison")

col1, col2, col3 = st.columns(3)

col1.metric(
    "Current Market Price",
    f"${current_price:.2f}"
)

col2.metric(
    "DCF Value",
    f"${intrinsic_value:.2f}"
)

if pd.isna(comps_implied_value):

    col3.metric(
        "Average Comps Value",
        "N/A"
    )

else:

    col3.metric(
        "Average Comps Value",
        f"${comps_implied_value:.2f}"
    )
```

```
    if not pd.isna(comps_implied_value):

        valuation_chart = go.Figure()

        valuation_chart.add_trace(
            go.Bar(
                x=[
                    "Market Price",
                    "DCF",
                    "Comparable Companies"
                ],
                y=[
                    current_price,
                    intrinsic_value,
                    comps_implied_value
                ],
                marker_color=[
                    NEON_PINK,
                    NEON_BLUE,
                    NEON_GREEN
                ]
            )
        )

        valuation_chart.update_layout(
            title="Market Price vs. DCF vs. Comparable Companies",
            xaxis_title="Valuation Method",
            yaxis_title="Value Per Share ($)",
            template="plotly_dark"
        )

        st.plotly_chart(
            valuation_chart,
            use_container_width=True
        )


##
## PART 4 - SENSITIVITY ANALYSIS
##

##

# Recalculate intrinsic value using different WACC
# and terminal growth assumptions.
def sensitivity_dcf(test_wacc, test_growth):

    projected = (
        base_fcf
        * (1 + forecast_growth) ** years
```

```
    )

    pv_projected = (
        projected
        / (1 + test_wacc) ** years
    )

    terminal = (
        projected[-1]
        * (1 + test_growth)
        / (test_wacc - test_growth)
    )

    pv_terminal_test = (
        terminal
        / (1 + test_wacc) ** forecast_years
    )

    enterprise_test = (
        pv_projected.sum()
        + pv_terminal_test
    )

    equity_test = (
        enterprise_test
        + cash
        - total_debt
    )

    return (
        equity_test
        / shares_outstanding
    )


##
## SENSITIVITY RANGES
##

wacc_values = [
    adjusted_wacc - 0.02,
    adjusted_wacc - 0.01,
    adjusted_wacc,
    adjusted_wacc + 0.01,
    adjusted_wacc + 0.02
]

terminal_growth_values = [
    max(0, terminal_growth - 0.01),
    max(0, terminal_growth - 0.005),
    terminal_growth,
    terminal_growth + 0.005,
```

```
    terminal_growth + 0.01
]


##
## BUILD SENSITIVITY TABLE
##

sensitivity_values = []

for test_wacc in wacc_values:

    row = []

    for test_growth in terminal_growth_values:

        if test_growth >= test_wacc:

            value = np.nan

        else:

            value = sensitivity_dcf(
                test_wacc,
                test_growth
            )

        row.append(value)

    sensitivity_values.append(row)


sensitivity_table = pd.DataFrame(
    sensitivity_values,
    index=[
        f"{value:.2%}"
        for value in wacc_values
    ],
    columns=[
        f"{value:.2%}"
        for value in terminal_growth_values
    ]
)

sensitivity_table.index.name = "WACC"
sensitivity_table.columns.name = "Terminal Growth"


sensitivity_min = np.nanmin(
    sensitivity_table.values
)
```

```
sensitivity_max = np.nanmax(
    sensitivity_table.values
)


##
## DETERMINE WHICH ASSUMPTION MOVES VALUE MORE
##

# Compare equal 1 percentage-point changes around the base case.
wacc_low_value = sensitivity_dcf(
    adjusted_wacc - 0.01,
    terminal_growth
)

wacc_high_value = sensitivity_dcf(
    adjusted_wacc + 0.01,
    terminal_growth
)

terminal_low_value = sensitivity_dcf(
    adjusted_wacc,
    max(0, terminal_growth - 0.01)
)

terminal_high_value = sensitivity_dcf(
    adjusted_wacc,
    terminal_growth + 0.01
)

wacc_value_range = abs(
    wacc_low_value - wacc_high_value
)

terminal_value_range = abs(
    terminal_high_value - terminal_low_value
)

if wacc_value_range > terminal_value_range:

    sensitivity_driver = "WACC"

else:

    sensitivity_driver = "Terminal Growth"


##
## DISPLAY PART 4
##

with sensitivity_tab:
```

```
st.subheader(
    f"{ticker} - DCF Sensitivity Analysis"
)

st.write(
    "Intrinsic Value Per Share: WACC vs. Terminal Growth"
)

st.info(
    f"Base Case: WACC {adjusted_wacc:.2%} | "
    f"Terminal Growth {terminal_growth:.2%} | "
    f"Intrinsic Value ${intrinsic_value:.2f}"
)


# Highlight the base-case sensitivity cell.
def highlight_base_case(data):

    styles = pd.DataFrame(
        "",
        index=data.index,
        columns=data.columns
    )

    base_row = f"{adjusted_wacc:.2%}"
    base_column = f"{terminal_growth:.2%}"

    if (
        base_row in styles.index
        and base_column in styles.columns
    ):

        styles.loc[
            base_row,
            base_column
        ] = (
            f"background-color: {NEON_PINK}; "
            "color: white; "
            "font-weight: bold;"
        )

    return styles


styled_sensitivity = (
    sensitivity_table.style
    .format("${:.2f}")
    .background_gradient(
        cmap="RdPu"
    )
    .apply(
```

```
        highlight_base_case,
        axis=None
    )
)

st.dataframe(
    styled_sensitivity
)


##
## HEATMAP
##

sensitivity_heatmap = go.Figure(
    data=go.Heatmap(
        z=sensitivity_table.values,
        x=sensitivity_table.columns,
        y=sensitivity_table.index,
        text=np.round(
            sensitivity_table.values,
            2
        ),
        texttemplate="$%{text}",
        colorscale=[
            [0.0, "#12001F"],
            [0.25, NEON_PURPLE],
            [0.5, NEON_PINK],
            [0.75, NEON_BLUE],
            [1.0, NEON_GREEN]
        ],
        colorbar=dict(
            title="Value / Share"
        )
    )
)

# Mark the base-case cell.
sensitivity_heatmap.add_trace(
    go.Scatter(
        x=[f"{terminal_growth:.2%}"],
        y=[f"{adjusted_wacc:.2%}"],
        mode="markers",
        marker=dict(
            symbol="square-open",
            size=45,
            color=NEON_PINK,
            line=dict(
                color=NEON_PINK,
                width=4
            )
        ),
```

```
        name="Base Case"
    )
)

sensitivity_heatmap.update_layout(
    title="DCF Sensitivity: WACC vs. Terminal Growth",
    xaxis_title="Terminal Growth",
    yaxis_title="WACC",
    template="plotly_dark"
)

st.plotly_chart(
    sensitivity_heatmap,
    use_container_width=True
)


##
## SENSITIVITY RANGE
##

st.write("Sensitivity Range")

col1, col2, col3 = st.columns(3)

col1.metric(
    "Lowest DCF Value",
    f"${sensitivity_min:.2f}"
)

col2.metric(
    "Base DCF Value",
    f"${intrinsic_value:.2f}"
)

col3.metric(
    "Highest DCF Value",
    f"${sensitivity_max:.2f}"
)


st.write("Sensitivity Interpretation")

st.write(
    f"Based on equal 1 percentage-point changes around "
    f"the base case, **{sensitivity_driver}** produces "
    f"the larger change in estimated intrinsic value."
)


if current_price < sensitivity_min:
```

```
        st.success(
            "The current market price is below the entire "
            "DCF sensitivity range. Under these assumptions, "
            "the shares appear inexpensive across the tested cases."
        )

    elif current_price > sensitivity_max:

        st.error(
            "The current market price is above the entire "
            "DCF sensitivity range. Under these assumptions, "
            "the shares appear expensive across the tested cases."
        )

    else:

        st.warning(
            "The current market price falls within the DCF "
            "sensitivity range, so the valuation conclusion "
            "depends on the assumptions used."
        )


##
## PART 5 — VALUATION SUMMARY

##

if implied_values.empty:

    comps_low = np.nan
    comps_high = np.nan

else:

    comps_low = implied_values.min()
    comps_high = implied_values.max()


week_52_low = safe_info(
    info,
    "fiftyTwoWeekLow"
)

week_52_high = safe_info(
    info,
    "fiftyTwoWeekHigh"
)


valuation_ranges = pd.DataFrame({
    "Method": [
```

```
        "DCF Sensitivity",
        "Comparable Companies",
        "52-Week Trading Range"
    ],
    "Low": [
        sensitivity_min,
        comps_low,
        week_52_low
    ],
    "High": [
        sensitivity_max,
        comps_high,
        week_52_high
    ]
})

valuation_ranges = valuation_ranges.dropna()


##
## DISPLAY PART 5
##

with extension_tab:

    st.subheader(
        f"{ticker} - Valuation Summary"
    )

    st.write("Key Valuation Results")

    col1, col2, col3 = st.columns(3)

    col1.metric(
        "Current Market Price",
        f"${current_price:.2f}"
    )

    col2.metric(
        "DCF Base Case",
        f"${intrinsic_value:.2f}"
    )

    if pd.isna(comps_implied_value):

        col3.metric(
            "Comparable Companies",
            "N/A"
        )

    else:
```

```
    col3.metric(
        "Comparable Companies",
        f"${comps_implied_value:.2f}"
    )


##
## FOOTBALL FIELD
##

st.write("Football Field Valuation")

football_chart = go.Figure()

football_colors = {
    "DCF Sensitivity": NEON_BLUE,
    "Comparable Companies": NEON_GREEN,
    "52-Week Trading Range": NEON_PURPLE
}

for _, row in valuation_ranges.iterrows():

    football_chart.add_trace(
        go.Scatter(
            x=[
                row["Low"],
                row["High"]
            ],
            y=[
                row["Method"],
                row["Method"]
            ],
            mode="lines+markers",
            line=dict(
                width=10,
                color=football_colors[row["Method"]]
            ),
            marker=dict(
                color=football_colors[row["Method"]],
                size=12
            ),
            name=row["Method"],
            showlegend=False
        )
    )


football_chart.add_vline(
    x=current_price,
    line_dash="dash",
    line_color=NEON_PINK,
    line_width=3,
```

```
    annotation_text="Current Market Price"
)


football_chart.update_layout(
    title="Valuation Range Comparison",
    xaxis_title="Value Per Share ($)",
    yaxis_title="Valuation Method",
    template="plotly_dark",
    height=500
)

st.plotly_chart(
    football_chart,
    use_container_width=True
)


st.write("Valuation Ranges")

st.dataframe(
    valuation_ranges.style.format({
        "Low": "${:.2f}",
        "High": "${:.2f}"
    }),
    hide_index=True
)


##
## FINAL VALUATION INTERPRETATION
##

st.write("Valuation Conclusion")

if pd.isna(comps_implied_value):

    st.warning(
        "Comparable-company data is incomplete, so the "
        "valuation conclusion should rely primarily on "
        "the DCF and sensitivity analysis."
    )

elif (
    current_price < intrinsic_value
    and current_price < comps_implied_value
):

    st.success(
        "Both the DCF and comparable-company analysis "
        "produce values above the current market price. "
        "Under the model assumptions, the shares appear undervalued."
```

```
        )

    elif (
        current_price > intrinsic_value
        and current_price > comps_implied_value
    ):

        st.error(
            "Both the DCF and comparable-company analysis "
            "produce values below the current market price. "
            "Under the model assumptions, the shares appear overvalued."
        )

    else:

        st.warning(
            "The DCF and comparable-company approaches produce "
            "different valuation conclusions. The result is therefore "
            "sensitive to the valuation method and assumptions used."
        )


    st.write("Model Interpretation")

    st.write(
        f"The base-case DCF estimates an intrinsic value of "
        f"${intrinsic_value:.2f} per share compared with a current "
        f"market price of ${current_price:.2f}. "
        f"The comparable-company approach produces an average "
        f"implied value of "
        f"{'$' + format(comps_implied_value, '.2f') if not pd.isna(comps_implied_va
        f"The DCF is particularly dependent on long-term assumptions "
        f"because terminal value represents {terminal_value_percent:.2f}% "
        f"of enterprise value. The sensitivity analysis identifies "
        f"{sensitivity_driver} as the larger valuation driver when "
        f"testing equal 1 percentage-point changes around the base case."
    )



##
## PART 6 - INVESTMENT ANALYSIS
##

with analysis_tab:

    st.subheader(
        f"{ticker} - Investment Analysis"
    )

    ##
    ## LULULEMON-SPECIFIC INVESTMENT ANALYSIS
```

```
##

if ticker == "LULU":

    st.info(
        "Can Lululemon's international growth offset weakness in "
        "the Americas and support a valuation above the current "
        "market price?"
    )

    ##
    ## GEOGRAPHIC REVENUE ANALYSIS
    ##

    st.write("Geographic Revenue Trends")

    # Geographic revenue comes from Lululemon's FY2025 Form 10-K.
    geographic_data = pd.DataFrame({
        "Region": [
            "Americas",
            "China Mainland",
            "Rest of World"
        ],
        "2023 Revenue ($B)": [
            7.63,
            0.96,
            1.02
        ],
        "2024 Revenue ($B)": [
            7.93,
            1.36,
            1.30
        ],
        "2025 Revenue ($B)": [
            7.85,
            1.75,
            1.50
        ],
        "2025 Growth": [
            -0.01,
            0.29,
            0.16
        ]
    })

    geographic_display = geographic_data.copy()

    geographic_display["2025 Growth"] = (
        geographic_display["2025 Growth"]
        .map(lambda x: f"{x:.1%}")
    )
```

```
st.dataframe(
    geographic_display,
    hide_index=True,
    use_container_width=True
)

##
## GEOGRAPHIC REVENUE CHART
##

geographic_chart = go.Figure()

geographic_chart.add_trace(
    go.Bar(
        name="2023",
        x=geographic_data["Region"],
        y=geographic_data["2023 Revenue ($B)"],
        marker_color=NEON_PINK
    )
)

geographic_chart.add_trace(
    go.Bar(
        name="2024",
        x=geographic_data["Region"],
        y=geographic_data["2024 Revenue ($B)"],
        marker_color=NEON_BLUE
    )
)

geographic_chart.add_trace(
    go.Bar(
        name="2025",
        x=geographic_data["Region"],
        y=geographic_data["2025 Revenue ($B)"],
        marker_color=NEON_GREEN
    )
)

geographic_chart.update_layout(
    title="Lululemon Revenue by Geographic Region",
    xaxis_title="Region",
    yaxis_title="Revenue ($ Billions)",
    barmode="group",
    template="plotly_dark"
)

st.plotly_chart(
    geographic_chart,
    use_container_width=True
)
```

```
##
## GEOGRAPHIC GROWTH
##

st.write("2025 Geographic Growth")

col1, col2, col3 = st.columns(3)

col1.metric(
    "Americas",
    "-1.0%"
)

col2.metric(
    "China Mainland",
    "+29.0%"
)

col3.metric(
    "Rest of World",
    "+16.0%"
)

##
## INTERNATIONAL OFFSET
##

americas_change = (
    geographic_data.loc[
        geographic_data["Region"] == "Americas",
        "2025 Revenue ($B)"
    ].iloc[0]
    -
    geographic_data.loc[
        geographic_data["Region"] == "Americas",
        "2024 Revenue ($B)"
    ].iloc[0]
)

china_change = (
    geographic_data.loc[
        geographic_data["Region"] == "China Mainland",
        "2025 Revenue ($B)"
    ].iloc[0]
    -
    geographic_data.loc[
        geographic_data["Region"] == "China Mainland",
        "2024 Revenue ($B)"
    ].iloc[0]
)

rest_world_change = (
```

```
    geographic_data.loc[
        geographic_data["Region"] == "Rest of World",
        "2025 Revenue ($B)"
    ].iloc[0]
    -
    geographic_data.loc[
        geographic_data["Region"] == "Rest of World",
        "2024 Revenue ($B)"
    ].iloc[0]
)

international_change = (
    china_change
    + rest_world_change
)

st.write("International Offset")

col1, col2 = st.columns(2)

col1.metric(
    "Americas Revenue Change",
    f"${americas_change:.2f}B"
)

col2.metric(
    "International Revenue Change",
    f"+${international_change:.2f}B"
)

st.success(
    "International growth more than offset the decline in "
    "Americas revenue. China Mainland and Rest of World added "
    f"approximately ${international_change:.2f}B of revenue, "
    f"while Americas revenue changed by ${americas_change:.2f}B."
)

##
## COMPANY FINANCIAL PERFORMANCE
##

st.write("Company Financial Performance")

latest_fundamentals = fundamentals_billions.iloc[-1]
previous_fundamentals = fundamentals_billions.iloc[-2]

latest_ratios = ratio_display.iloc[-1]
previous_ratios = ratio_display.iloc[-2]

latest_year = fundamentals_billions.index[-1]
previous_year = fundamentals_billions.index[-2]
```

```
col1, col2, col3, col4 = st.columns(4)

col1.metric(
    f"{latest_year} Revenue",
    f"${latest_fundamentals['Revenue']:.2f}B",
    f"${latest_fundamentals['Revenue'] - previous_fundamentals['Revenue']:.
)

col2.metric(
    f"{latest_year} Operating Income",
    f"${latest_fundamentals['Operating Income']:.2f}B",
    f"${latest_fundamentals['Operating Income'] - previous_fundamentals['Op
)

col3.metric(
    f"{latest_year} Free Cash Flow",
    f"${latest_fundamentals['Free Cash Flow']:.2f}B",
    f"${latest_fundamentals['Free Cash Flow'] - previous_fundamentals['Free
)

col4.metric(
    f"{latest_year} Operating Margin",
    f"{latest_ratios['Operating Margin']:.2f}%",
    f"{latest_ratios['Operating Margin'] - previous_ratios['Operating Margi
)

##
## OPERATING INTERPRETATION
##

st.write("Operating Interpretation")

st.write(
    f"Total company revenue increased from "
    f"${previous_fundamentals['Revenue']:.2f}B in {previous_year} "
    f"to ${latest_fundamentals['Revenue']:.2f}B in {latest_year}. "
    f"This supports the idea that international growth is helping "
    f"offset weakness in the Americas."
)

st.write(
    f"However, operating income changed from "
    f"${previous_fundamentals['Operating Income']:.2f}B to "
    f"${latest_fundamentals['Operating Income']:.2f}B, while "
    f"free cash flow changed from "
    f"${previous_fundamentals['Free Cash Flow']:.2f}B to "
    f"${latest_fundamentals['Free Cash Flow']:.2f}B. "
    f"Operating margin also changed from "
    f"{previous_ratios['Operating Margin']:.2f}% to "
    f"{latest_ratios['Operating Margin']:.2f}%."
)
```

```
st.warning(
    "International expansion is currently offsetting Americas "
    "weakness at the revenue level, but weaker profitability and "
    "free cash flow show that it has not fully offset the economic "
    "impact of the slowdown."
)

##
## VALUATION EVIDENCE
##

st.write("Valuation Evidence")

col1, col2, col3 = st.columns(3)

col1.metric(
    "Current Market Price",
    f"${current_price:.2f}"
)

col2.metric(
    "DCF Intrinsic Value",
    f"${intrinsic_value:.2f}"
)

if pd.isna(comps_implied_value):

    col3.metric(
        "Comparable Companies",
        "N/A"
    )

else:

    col3.metric(
        "Comparable Companies",
        f"${comps_implied_value:.2f}"
    )

##
## INVESTMENT CONCLUSION
##

st.write("Investment Conclusion")

if (
    not pd.isna(comps_implied_value)
    and intrinsic_value > current_price
    and comps_implied_value > current_price
):

    st.success(
```

```
            f"The evidence partially supports the investment hypothesis. "
            f"International growth has more than offset the decline in "
            f"Americas revenue, allowing total company revenue to continue "
            f"growing. However, declining operating performance and free "
            f"cash flow show that international growth has not completely "
            f"offset the underlying pressure on the business. The DCF "
            f"estimates ${intrinsic_value:.2f} per share and the comparable "
            f"company analysis estimates ${comps_implied_value:.2f}, both "
            f"above the current market price of ${current_price:.2f}. "
            f"Under the model assumptions, the market therefore appears "
            f"to be pricing in a more severe or prolonged slowdown than "
            f"the base-case valuation assumes."
        )

    elif intrinsic_value > current_price:

        st.info(
            f"International growth is offsetting weakness in Americas "
            f"revenue, although profitability and free cash flow remain "
            f"under pressure. The DCF value of ${intrinsic_value:.2f} "
            f"is above the current market price of ${current_price:.2f}, "
            f"but comparable-company evidence is incomplete."
        )

    else:

        st.warning(
            "International growth is helping offset Americas weakness, "
            "but the valuation analysis does not currently indicate "
            "sufficient upside to conclude that the market is overly "
            "pessimistic."
        )

    ##
    ## KEY VALUATION RISK
    ##

    st.write("Key Valuation Risk")

    st.write(
        f"Terminal value represents {terminal_value_percent:.2f}% of "
        f"enterprise value in the base-case DCF. This means the valuation "
        f"depends heavily on long-term assumptions. The sensitivity "
        f"analysis identifies {sensitivity_driver} as the larger valuation "
        f"driver when testing equal 1 percentage-point changes."
    )

    st.caption(
        "Geographic revenue data: Lululemon FY2025 Form 10-K. "
        "Financial performance and valuation figures are calculated "
        "from the dashboard."
    )
```

```
##
## OTHER TICKERS - DYNAMIC INVESTMENT ANALYSIS
##

else:

    st.info(
        f"Does {ticker}'s recent financial performance support "
        f"the valuation implied by the DCF and comparable-company analysis?"
    )

    ##
    ## FINANCIAL PERFORMANCE
    ##

    st.write("Financial Performance")

    latest_fundamentals = fundamentals_billions.iloc[-1]
    previous_fundamentals = fundamentals_billions.iloc[-2]

    latest_ratios = ratio_display.iloc[-1]
    previous_ratios = ratio_display.iloc[-2]

    latest_year = fundamentals_billions.index[-1]
    previous_year = fundamentals_billions.index[-2]

    revenue_change = (
        latest_fundamentals["Revenue"]
        - previous_fundamentals["Revenue"]
    )

    operating_income_change = (
        latest_fundamentals["Operating Income"]
        - previous_fundamentals["Operating Income"]
    )

    fcf_change = (
        latest_fundamentals["Free Cash Flow"]
        - previous_fundamentals["Free Cash Flow"]
    )

    operating_margin_change = (
        latest_ratios["Operating Margin"]
        - previous_ratios["Operating Margin"]
    )

    col1, col2, col3, col4 = st.columns(4)

    col1.metric(
        f"{latest_year} Revenue",
        f"${latest_fundamentals['Revenue']:.2f}B",
```

```
    f"${revenue_change:.2f}B"
)

col2.metric(
    f"{latest_year} Operating Income",
    f"${latest_fundamentals['Operating Income']:.2f}B",
    f"${operating_income_change:.2f}B"
)

col3.metric(
    f"{latest_year} Free Cash Flow",
    f"${latest_fundamentals['Free Cash Flow']:.2f}B",
    f"${fcf_change:.2f}B"
)

col4.metric(
    f"{latest_year} Operating Margin",
    f"{latest_ratios['Operating Margin']:.2f}%",
    f"{operating_margin_change:.2f} pts"
)

##
## FINANCIAL TREND CHART
##

st.write("Recent Financial Trends")

analysis_chart = go.Figure()

analysis_chart.add_trace(
    go.Bar(
        name="Revenue",
        x=fundamentals_billions.index.astype(str),
        y=fundamentals_billions["Revenue"],
        marker_color=NEON_PINK
    )
)

analysis_chart.add_trace(
    go.Bar(
        name="Operating Income",
        x=fundamentals_billions.index.astype(str),
        y=fundamentals_billions["Operating Income"],
        marker_color=NEON_BLUE
    )
)

analysis_chart.add_trace(
    go.Bar(
        name="Free Cash Flow",
        x=fundamentals_billions.index.astype(str),
        y=fundamentals_billions["Free Cash Flow"],
```

```
        marker_color=NEON_GREEN
    )
)

analysis_chart.update_layout(
    title=f"{ticker} Financial Performance",
    xaxis_title="Fiscal Year",
    yaxis_title="$ Billions",
    barmode="group",
    template="plotly_dark",
    xaxis=dict(type="category")
)

st.plotly_chart(
    analysis_chart,
    use_container_width=True
)

##
## OPERATING INTERPRETATION
##

if revenue_change > 0:

    revenue_direction = "increased"

elif revenue_change < 0:

    revenue_direction = "decreased"

else:

    revenue_direction = "was unchanged"


if operating_income_change > 0:

    operating_direction = "increased"

elif operating_income_change < 0:

    operating_direction = "decreased"

else:

    operating_direction = "was unchanged"


if fcf_change > 0:

    fcf_direction = "increased"
```

```
elif fcf_change < 0:

    fcf_direction = "decreased"

else:

    fcf_direction = "was unchanged"


st.write(
    f"Revenue {revenue_direction} by "
    f"${abs(revenue_change):.2f}B from {previous_year} to "
    f"{latest_year}. Operating income {operating_direction} by "
    f"${abs(operating_income_change):.2f}B, while free cash flow "
    f"{fcf_direction} by ${abs(fcf_change):.2f}B."
)

st.write(
    f"Operating margin changed from "
    f"{previous_ratios['Operating Margin']:.2f}% in "
    f"{previous_year} to "
    f"{latest_ratios['Operating Margin']:.2f}% in "
    f"{latest_year}."
)


##
## VALUATION EVIDENCE
##

st.write("Valuation Evidence")

col1, col2, col3 = st.columns(3)

col1.metric(
    "Current Market Price",
    f"${current_price:.2f}"
)

col2.metric(
    "DCF Intrinsic Value",
    f"${intrinsic_value:.2f}"
)

if pd.isna(comps_implied_value):

    col3.metric(
        "Comparable Companies",
        "N/A"
    )

else:
```

```
    col3.metric(
        "Comparable Companies",
        f"${comps_implied_value:.2f}"
    )


##
## INVESTMENT CONCLUSION
##

st.write("Investment Conclusion")

if (
    not pd.isna(comps_implied_value)
    and intrinsic_value > current_price
    and comps_implied_value > current_price
):

    st.success(
        f"Both valuation approaches estimate a value above "
        f"the current market price. The DCF estimates "
        f"${intrinsic_value:.2f} per share and the comparable "
        f"company analysis estimates ${comps_implied_value:.2f}, "
        f"compared with a current market price of "
        f"${current_price:.2f}. Under the model assumptions, "
        f"the shares appear undervalued."
    )

elif (
    not pd.isna(comps_implied_value)
    and intrinsic_value < current_price
    and comps_implied_value < current_price
):

    st.error(
        f"Both valuation approaches estimate a value below "
        f"the current market price. The DCF estimates "
        f"${intrinsic_value:.2f} per share and the comparable "
        f"company analysis estimates ${comps_implied_value:.2f}, "
        f"compared with a current market price of "
        f"${current_price:.2f}. Under the model assumptions, "
        f"the shares appear overvalued."
    )

elif pd.isna(comps_implied_value):

    if intrinsic_value > current_price:

        st.info(
            f"The DCF estimates an intrinsic value of "
            f"${intrinsic_value:.2f} per share, above the "
```

```
            f"current market price of ${current_price:.2f}. "
            f"Comparable-company data is incomplete, so the "
            f"investment conclusion relies primarily on the DCF."
        )

    else:

        st.info(
            f"The DCF estimates an intrinsic value of "
            f"${intrinsic_value:.2f} per share, below the "
            f"current market price of ${current_price:.2f}. "
            f"Comparable-company data is incomplete, so the "
            f"investment conclusion relies primarily on the DCF."
        )

else:

    st.warning(
        f"The DCF and comparable-company approaches produce "
        f"different conclusions relative to the current market "
        f"price of ${current_price:.2f}. The investment conclusion "
        f"is therefore sensitive to the valuation method used."
    )


##
## KEY VALUATION RISK
##

st.write("Key Valuation Risk")

st.write(
    f"Terminal value represents {terminal_value_percent:.2f}% "
    f"of enterprise value in the base-case DCF. As a result, "
    f"the valuation depends heavily on long-term assumptions. "
    f"The sensitivity analysis identifies {sensitivity_driver} "
    f"as the larger valuation driver when testing equal "
    f"1 percentage-point changes around the base case."
)
```